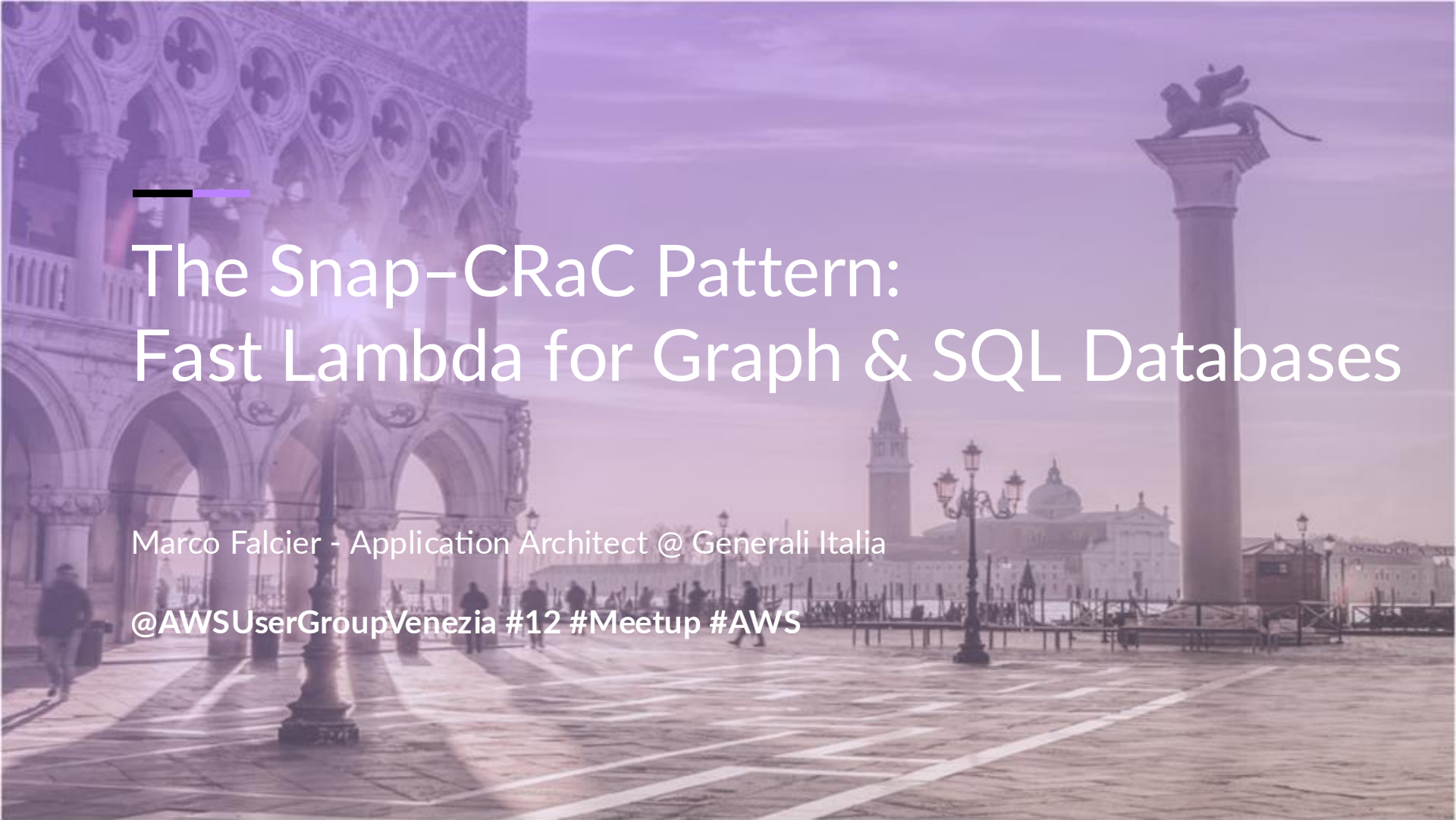




The Snap-CRaC Pattern: Fast Lambda for Graph & SQL Databases

Marco Falcier - Application Architect @ Generali Italia

@AWSUserGroupVenezia #12 #Meetup #AWS

Who am I?

Application Architect @ Generali

- Serverless Enthusiast
- Kotlin Lover
- Neo4j Certified Professional

 @mfalcier

 mfalcier

 marcofalcier





In This Talk...

We'll explore a practical pattern to build fast AWS Lambda functions that safely interact with relational and graph databases, using SnapStart and CRaC to control initialization and lifecycle.



The Problem: Cold Starts and Databases

Lambda is fast... until it touches a database

- Cold start = JVM Boot + Optional Inits + DB Setup
- Relational & Graph DBs amplify the issue

Key Message: DB initialization dominates cold start time



Traditional Mitigations (and Limits)

- Provisioned Concurrency
- Smaller dependencies / keep-alive hacks

Limits: cost, partial fixes, slow init



Why Databases Are Special

Not all resources behave the same

- DB connections are:
 - Expensive to create
 - Stateful
 - Often pooled
- Graph DB drivers often perform heavy startup work

Implication: naive Lambda init hurts performance



Enter Snap

What Snapstart Does

- Initializes Lambda once
- Takes a snapshot of memory & execution state
- Restores it on every cold start

Result: JVM cold start $\approx$ milliseconds



Enter CRaC

CRaC: Coordinated Restore at Checkpoint

- Hooks into JVM lifecycle
- Prepare / restore resources safely

Perfect match for SnapStart



The Snap-CRaC Pattern

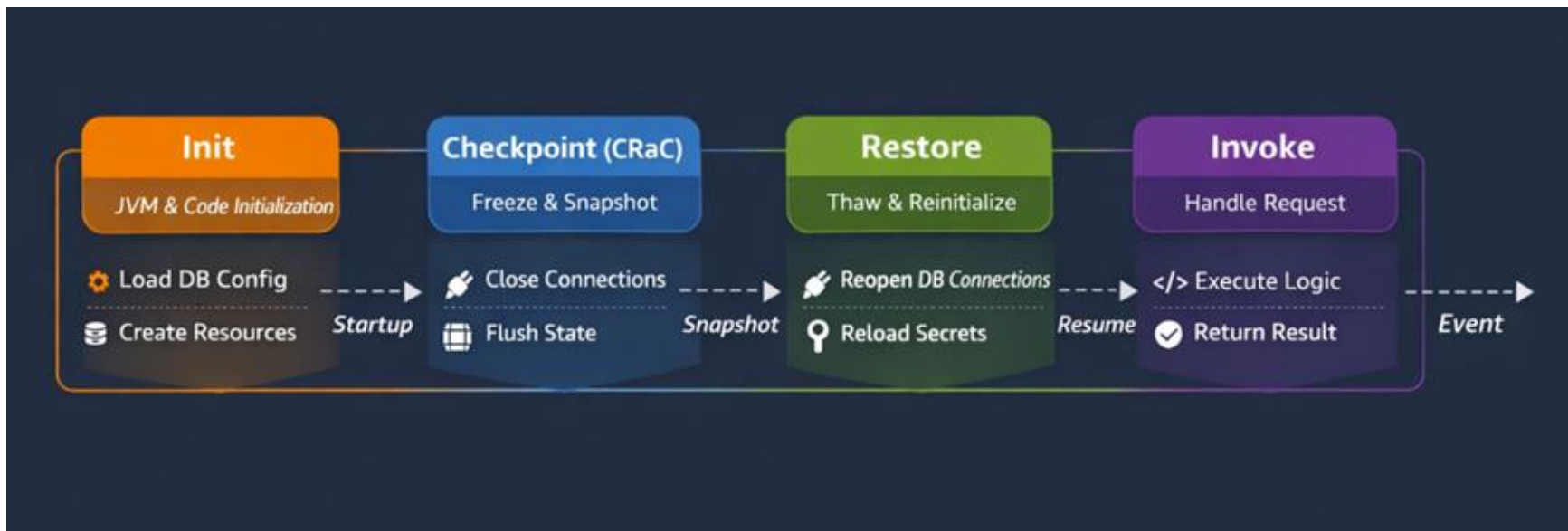
Combining SnapStart + CRaC

Pattern steps:

1. Initialize drivers & config once
2. Close unsafe resources at checkpoint
3. Restore connections on resume

Goal: fast start + safe DB state

Lifecycle View





SQL vs Graph DB Considerations

SQL

- JDBC + connection pools (e.g. HikariCP)
- Pools must be recreated on restore

Graph

- Heavy driver initialization
- Metadata & routing cached once

Why it matters: graph drivers benefits massively from SnapStart



What You Should NOT Snapshot

- Open sockets
- Active transactions
- Thread pools

Rule: snapshot logic, not live connections



Performance Impact

- Cold start: seconds → milliseconds
- Stable latency under burst traffic
- Lower cost vs Provisioned Concurrency

Real impact: predictable p99 latency



When NOT to use This Pattern

- Non-Java runtimes
- Highly dynamic or per-request initialization
- Workloads without cold-start sensitivity

SnapStart ≠ universal solution

CRaC Resource Example

```
public class CRaCResource implements Resource { no usages

    public CRaCResource () { no usages
        Core.getGlobalContext().register( R, this);
    }

    private void refreshDriver() { 1 usage
        System.out.println("CRaCResource.refreshDriver()");
    }

    private void closeDriver() { 1 usage
        System.out.println("CRaCResource.closeDriver()");
    }

    public void executeQuery() { no usages
        System.out.println("CRaCResource.executeQuery()");
    }

    @Override public void beforeCheckpoint(Context<? extends Resource> c) { no usages
        closeDriver();
    }

    @Override public void afterRestore(Context<? extends Resource> c) { no usages
        refreshDriver();
    }

}
```

Example in CloudWatch

The screenshot displays a list of log entries from an AWS CloudWatch console. The entries are grouped into three distinct sections, each marked by a colored bracket on the left side of the log lines:

- Red Bracket (INIT):** This group includes the first three log entries, which correspond to the initialization phase of a request. The log messages are:
 - `INIT_START Runtime Version: java:21.v56 Runtime Version ARN: arn_...`
 - `CRaCResource.beforeCheckpoint()`
 - `CRaCResource.closeDriver()`
- Yellow Bracket (RESTORE):** This group includes the next four log entries, corresponding to the restore phase. The log messages are:
 - `INIT_REPORT Init Duration: 1832.99 ms`
 - `RESTORE_START Runtime Version: java:21.v56 Runtime Version ARN: _...`
 - `CRaCResource.afterRestore()`
 - `CRaCResource.refreshDriver()`
- Green Bracket (REPORT):** This group includes the final three log entries, corresponding to the report phase. The log messages are:
 - `RESTORE_REPORT Restore Duration: 794.81 ms`
 - `START RequestId: ee34d62b-e3f9-4468-9d2d-c0a41afb0586 Version: 13`
 - `END RequestId: ee34d62b-e3f9-4468-9d2d-c0a41afb0586`

The log entries continue with a `REPORT` message for the first request, followed by a new `START` message for a second request (RequestId: e5cca386-9027-45a7-9eba-4aafd926f4a4), and finally its `END` and `REPORT` messages.

Neptune Resource Example

```
@Getter 3 usages
public class NeptuneResource implements Resource {

    private Session session;

    public NeptuneResource() { 1 usage
        Core.getGlobalContext().register(this);
        this.session = NeptuneConfig.driver().session();
        System.out.println("NeptuneResource costruttore ha creato");
    }

    public void executeQuery() { 1 usage
        // implement some logic here
    }

    @Override no usages
    public void beforeCheckpoint(Context<? extends Resource> context) throws Exception {
        System.out.println("Before checkpoint: closing NeptuneResource.session");
        this.session.close();
    }

    @Override no usages
    public void afterRestore(Context<? extends Resource> context) throws Exception {
        System.out.println("After restore: refreshing NeptuneResource.session");
        this.session = NeptuneConfig.driver().session();
    }
}
```



Key Takeaways

- SnapStart removes JVM startup cost
- CRaC makes DB access snapshot-safe
- Works for both SQL & Graph DBs
- Enables truly fast serverless architecture



Closing

Cold starts aren't the problem,
uncontrolled initialization is.



Questions ?

@AWSUserGroupVenezia #4 #Meetup #AWS